

Déployer Airflow sur son propre EC2

Même principe de sécurité que le pipeline de `myapp` — secret dédié, rôle IAM scopé, aucune clé statique — appliqué à un EC2 séparé qui héberge Airflow en Celery (webserver, scheduler, worker, Postgres, Redis).

POURQUOI UN DOCUMENT ET UN WORKFLOW SÉPARÉS

Airflow tourne sur un EC2 différent, avec sa propre image et son propre cycle de vie — un changement dans `app/` ne doit jamais redéployer Airflow, et inversement. Deux workflows indépendants, chacun avec son rôle IAM, son secret et son déclencheur (`paths:` scopé à `airflow/`), donnent une isolation nette et un historique de déploiement propre à chaque service — plutôt qu'un seul pipeline qui finit par tout mélanger.

1 Secret

2 Rôle IAM EC2

3 `deploy.sh`

4 `docker-compose`

5 Rôle OIDC

6 Workflow

7 Checklist

1 Le secret Airflow dans Secrets Manager

Un secret séparé de celui de `myapp`, qui ne contient que ce qui est réellement sensible dans votre `.env` actuel. Tout le reste (bucket, préfixes, région, intervalle) n'a pas sa place ici — voir l'étape 4.

BASH

```
aws secretsmanager create-secret \
  --name airflow/prod \
  --region eu-west-3 \
  --secret-string '{
    "AIRFLOW_FERNET_KEY": "xxxxxx",
    "_AIRFLOW_WWW_USER_USERNAME": "airflow",
    "_AIRFLOW_WWW_USER_PASSWORD": "yyyyyy"
  }'
```

GARDEZ VOS NOMS DE VARIABLES EXACTS

`deploy.sh` transforme chaque clé du secret directement en variable d'environnement, sans renommer quoi que ce soit. Les noms ci-dessus doivent donc être *exactement* ceux que votre `docker-compose.yml` attend déjà — ici `AIRFLOW_FERNET_KEY` et non `AIRFLOW__CORE__FERNET_KEY`, parce que c'est le nom utilisé dans votre fichier actuel.

`AIRFLOW_FERNET_KEY` chiffre les mots de passe des connexions stockées par Airflow — générez-le une fois (`python3 -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"`) et ne le changez plus ensuite, sous peine de perdre l'accès aux connexions déjà chiffrées. `_AIRFLOW_WWW_USER_USERNAME` n'est pas sensible en soi, mais autant le garder avec le mot de passe puisqu'il en forme la paire.

2 Rôle IAM de l'instance EC2 Airflow

Même schéma que pour `myapp`, plus un droit que `myapp` n'avait pas besoin : l'accès au bucket S3 utilisé pour synchroniser DAGs, données et logs (`S3_BUCKET`, `DAGS_S3_PREFIX`, etc. de votre `.env`). Toujours un rôle distinct de celui de `myapp`, sans accès croisé entre les deux secrets.

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadSecret",
      "Effect": "Allow",
      "Action": "secretsmanager:GetSecretValue",
      "Resource": "arn:aws:secretsmanager:eu-west-3:123456789012:secret:airflow/pr",
    },
    {
      "Sid": "EcrAuth",
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*"
    },
    {
      "Sid": "EcrPull",
      "Effect": "Allow",
      "Action": ["ecr:BatchGetImage", "ecr:GetDownloadUrlForLayer"],
```

```

    "Resource": "arn:aws:ecr:eu-west-3:123456789012:repository/airflow"
  },
  {
    "Sid": "DagsDataLogsSync",
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject", "s3:ListBucket"],
    "Resource": [
      "arn:aws:s3:::nappecast",
      "arn:aws:s3:::nappecast/airflow/*"
    ]
  }
]
}

```

La statement `DagsDataLogsSync` mélange lecture et écriture par simplicité — resserrez-la si vous savez déjà que, par exemple, `DAGS_S3_PREFIX` n'est jamais qu'en lecture depuis l'instance (les DAGs y sont déposés par un autre processus) : dans ce cas, un `GetObject` / `ListBucket` scopé à ce seul préfixe suffit, et seul `LOGS_S3_PREFIX` a vraiment besoin de `PutObject`.

3 deploy.sh, versionné dans Git

Même logique que pour `myapp` : ce fichier vit dans `airflow/deploy.sh` du dépôt et arrive par SCP à chaque déploiement (étape 6) — jamais modifié à la main sur l'instance.

```

BASH - /OPT/AIRFLOW/DEPLOY.SH

#!/usr/bin/env bash
set -euo pipefail

APP_DIR=/opt/airflow
ENV_FILE="$APP_DIR/.env.runtime"
SECRET_ID="airflow/prod"
REGION="eu-west-3"
ECR_REGISTRY="123456789012.dkr.ecr.eu-west-3.amazonaws.com"

umask 077
aws secretsmanager get-secret-value \
  --secret-id "$SECRET_ID" \
  --region "$REGION" \
  --query SecretString --output text \
  | jq -r 'to_entries[] | "\(.key)=\(.value)"' > "$ENV_FILE"

chmod 600 "$ENV_FILE"

```

```
chown appuser:appuser "$ENV_FILE" "$APP_DIR/docker-compose.yml" "$APP_DIR/.env"

aws ecr get-login-password --region "$REGION" \
  | docker login --username AWS --password-stdin "$ECR_REGISTRY"

cd "$APP_DIR"
docker compose pull
docker compose --env-file "$ENV_FILE" up -d
docker image prune -f
```

`deploy.sh` ne touche jamais au `.env` non-secret (livré à côté par SCP, étape 6) — il ne fait que le rendre lisible par `appuser`, sans jamais le générer ni le modifier.

User Data de l'instance (provisionnement initial)

Identique dans l'esprit au guide `myapp` : installe Docker, crée l'utilisateur applicatif et `/opt/airflow`. N'écrit ni `deploy.sh`, ni `docker-compose.yml`, ni `.env` — les trois arrivent par SCP au premier déploiement (étape 6).

```
BASH – USER DATA (AMAZON LINUX 2023)

#!/bin/bash
set -euo pipefail

dnf update -y
dnf install -y docker jq
systemctl enable --now docker

mkdir -p /usr/local/lib/docker/cli-plugins
curl -sSL https://github.com/docker/compose/releases/latest/download/docker-compose
  -o /usr/local/lib/docker/cli-plugins/docker-compose
chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

useradd -r -s /usr/sbin/nologin -d /opt/airflow appuser || true
usermod -aG docker appuser
mkdir -p /opt/airflow
chown appuser:appuser /opt/airflow
```

Le rôle IAM (étape 2) doit être attaché à l'instance au lancement, comme pour `myapp`.

4 docker-compose.yml : brancher les deux fichiers d'environnement

Vous avez déjà un `Dockerfile` et un `docker-compose.yml` fonctionnels pour Airflow — inutile de les récrire. Le seul changement à y apporter : chaque service lit désormais **deux** `env_file` au lieu d'un seul `.env` — le vôtre, non-secret, et `.env.runtime`, généré par `deploy.sh` à partir du secret.

YAML – /OPT/AIRFLOW/DOCKER-COMPOSE.YML (EXTRAIT)

```
x-airflow-common: &airflow-common
  image: 123456789012.dkr.ecr.eu-west-3.amazonaws.com/airflow:latest
  env_file:
    - .env          # AIRFLOW_UID, S3_BUCKET, DAGS_S3_PREFIX, DATA_S3_PREFIX,
                    # LOGS_S3_PREFIX, AWS_REGION, SYNC_INTERVAL_SECONDS
    - .env.runtime  # AIRFLOW_FERNET_KEY, _AIRFLOW_WWW_USER_USERNAME,
                    # _AIRFLOW_WWW_USER_PASSWORD

services:
  airflow-webserver:
    <<: *airflow-common
    command: webserver
    ports: ["8080:8080"]
    restart: unless-stopped

  airflow-scheduler:
    <<: *airflow-common
    command: scheduler
    restart: unless-stopped

  dags-sync:
    <<: *airflow-common
    command: ["/sync.sh"]
    restart: unless-stopped
```

Le fichier `.env` ci-dessus est celui que vous avez déjà — il n'a besoin d'aucune modification, juste d'être livré tel quel sur l'instance (étape 6). Seules les trois valeurs réellement sensibles quittent votre `.env` pour rejoindre le secret (étape 1) ; le reste continue de vivre en clair, versionné dans Git comme aujourd'hui.

5 Rôle OIDC dédié pour GitHub Actions

Le fournisseur OIDC (`token.actions.githubusercontent.com`) est déjà créé une fois pour tout le compte — inutile de le recréer. Il faut en revanche un **second rôle**, distinct de `github-actions-deploy`, avec ses propres permissions ECR limitées au repo `airflow`.

JSON – TRUST POLICY

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.github",
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
        },
        "StringLike": {
          "token.actions.githubusercontent.com:sub": "repo:mon-org/mon-repo:ref:re"
        }
      }
    }
  ]
}
```

Même dépôt, même branche que le rôle `myapp` (monorepo oblige) — c'est la policy de **permissions** qui isole les deux rôles, pas la trust policy : `ecr:PutImage` et consorts scopés à `arn:aws:ecr:eu-west-3:123456789012:repository/airflow` uniquement, sans aucun droit sur le repo de `myapp` ni sur Secrets Manager.

BASH

```
aws iam create-role \
  --role-name github-actions-deploy-airflow \
  --assume-role-policy-document file://trust-policy.json

aws iam put-role-policy \
  --role-name github-actions-deploy-airflow \
  --policy-name github-actions-ecr-airflow \
  --policy-document file://permissions-policy.json
```

6 Le workflow dédié, filtré sur airflow/

Le déclencheur `paths:` garantit qu'un commit qui ne touche que `app/` ne redéploie jamais Airflow, et vice-versa.

YAML - .GITHUB/WORKFLOWS/DEPLOY-AIRFLOW.YML

name: Deploy Airflow

on:

push:

branches: [main]

paths:

- 'airflow/**'

permissions:

id-token: write

contents: read

jobs:

build-and-deploy:

runs-on: ubuntu-latest

environment: production

steps:

- uses: actions/checkout@v4

- name: Auth AWS via OIDC

uses: aws-actions/configure-aws-credentials@v4

with:

role-to-assume: arn:aws:iam::123456789012:role/github-actions-deploy-air

aws-region: eu-west-3

- name: Login ECR

uses: aws-actions/amazon-ecr-login@v2

- name: Build et push de l'image

uses: docker/build-push-action@v6

with:

context: ./airflow

push: true

tags: 123456789012.dkr.ecr.eu-west-3.amazonaws.com/airflow:\${{ github.sh

cache-from: type=gha

cache-to: type=gha,mode=max

- name: Rendre le docker-compose.yml avec le tag de cette build

run: |

sed -i "s|image: 123456789012.dkr.ecr.eu-west-3.amazonaws.com/airflow:.*

- name: Copier docker-compose.yml, .env et deploy.sh sur l'EC2 (SCP, aucun s

uses: appleboy/scp-action@v0.1.7

with:

host: \${ secrets.AIRFLOW_EC2_HOST }}

username: \${ secrets.AIRFLOW_EC2_USER }}

key: \${ secrets.AIRFLOW_EC2_SSH_KEY }}

```

source: "airflow/docker-compose.yml,airflow/.env,airflow/deploy.sh"
strip_components: 1
target: "~"

- name: Déploiement (SSH, aucun secret transmis)
  uses: appleboy/ssh-action@v1
  with:
    host: ${ secrets.AIRFLOW_EC2_HOST }}
    username: ${ secrets.AIRFLOW_EC2_USER }}
    key: ${ secrets.AIRFLOW_EC2_SSH_KEY }}
    script: |
      sudo mv ~/docker-compose.yml /opt/airflow/docker-compose.yml
      sudo mv ~/.env /opt/airflow/.env
      sudo mv ~/deploy.sh /opt/airflow/deploy.sh
      sudo chmod +x /opt/airflow/deploy.sh
      sudo chown appuser:appuser /opt/airflow/deploy.sh
      sudo /opt/airflow/deploy.sh

```

SECRETS GITHUB DISTINCTS

`AIRFLOW_EC2_HOST` , `AIRFLOW_EC2_USER` , `AIRFLOW_EC2_SSH_KEY` sont des identifiants d'accès à cette instance, séparés de `EC2_HOST` / `EC2_USER` / `EC2_SSH_KEY` utilisés pour `myapp` — une clé compromise ne donne accès qu'à un seul serveur. Le `.env` transmis ici ne contient, par construction, aucune des trois valeurs sensibles (étape 1) : elles ne quittent jamais Secrets Manager avant d'atteindre l'instance elle-même.

7 Checklist sécurité

- ✓ **Secret, rôle IAM EC2 et rôle OIDC distincts** de ceux de `myapp` — aucun accès croisé entre les deux services.
- ✓ **Filtre** `paths: airflow/**` sur le déclencheur — vérifié en poussant un commit qui ne touche que `app/` et en confirmant qu'aucun déploiement Airflow ne se déclenche.
- ✓ **FERNET_KEY et WEBSERVER__SECRET_KEY générées une fois**, jamais régénérées après la première mise en production (les connexions Airflow existantes deviendraient illisibles).
- ✓ **Clé SSH de déploiement dédiée** à cette instance (`AIRFLOW_EC2_SSH_KEY`), distincte de celle de `myapp` .

✓ **Port 8080 (webserver)** restreint par groupe de sécurité — pas ouvert au monde entier.

✓ **Tag d'image immuable** (`github.sha`), jamais `:latest` en production.

✓ **Relecture du `.env` avant chaque commit** — c'est un fichier en clair, versionné dans Git : vérifier qu'aucune valeur sensible n'y a été ajoutée par erreur au fil du temps.

✓ **Permissions S3 resserrées** au strict nécessaire par préfixe (lecture pour les DAGs/données, écriture seulement là où c'est réellement utile), pas un accès lecture/écriture large sur tout le bucket.

Compte (`123456789012`), région (`eu-west-3`) et dépôt (`mon-org/mon-repo`) sont des exemples à remplacer par les vôtres — voir le guide « Pipeline Secrets AWS » pour le détail des mécanismes partagés (OIDC, permissions vs trust policy, User Data).